

Observability

Table of Content

- Monitoring vs. Observability
- Caveats of “The Old Ways” of testing ML models
- An overview of ways to test your LLM

Table of Content

- **Monitoring vs. Observability**
- Caveats of “The Old Ways” of testing ML models
- An overview of ways to test your LLM

Monitoring: The "What"

Focus: Tracking health, alerting, and identifying *what* happened.

Key Indicators: Latency, throughput, and error rate + guardrails



TTF

profile

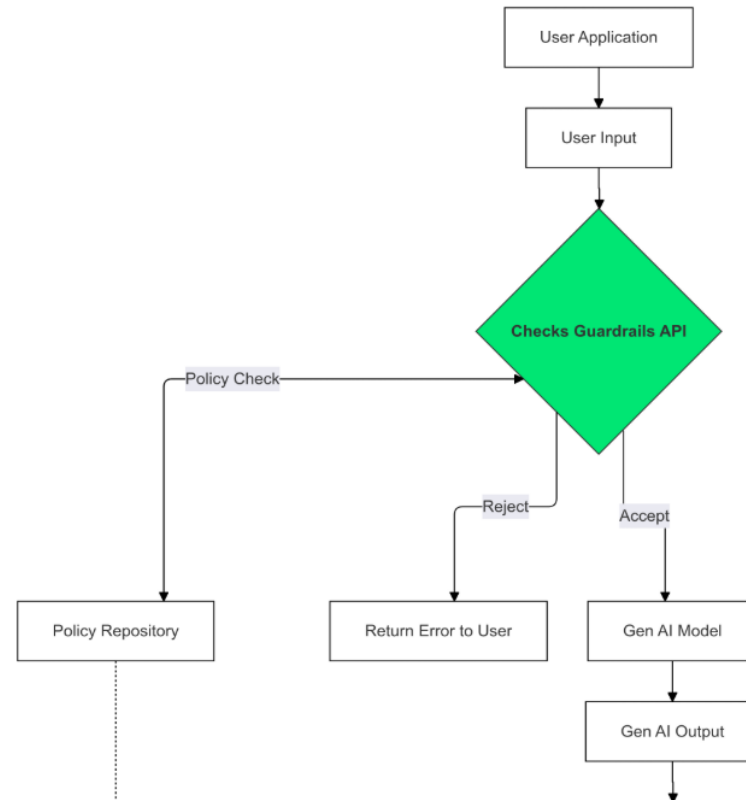
margin brechen

Observability: The "Why"

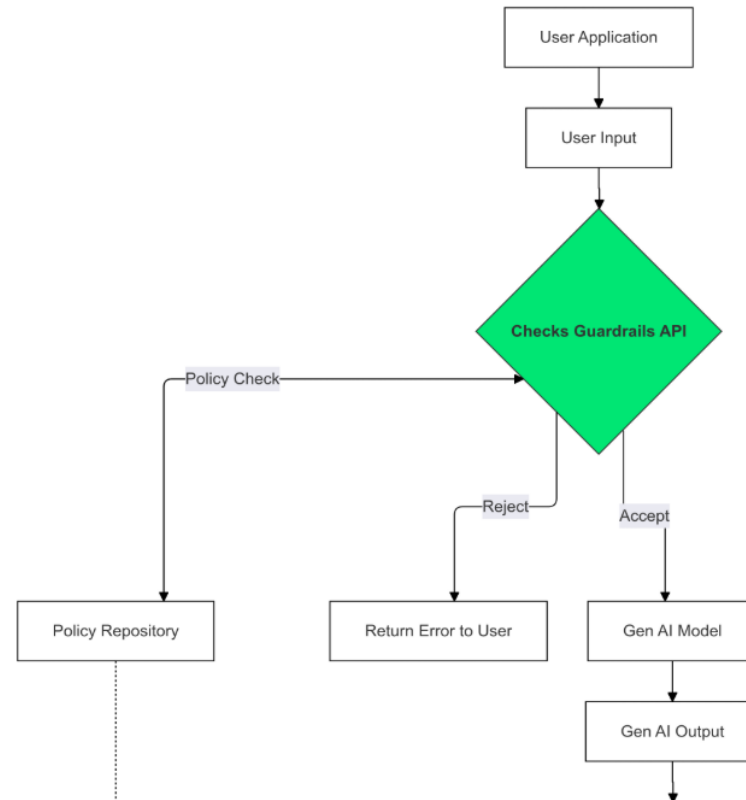
Focus: Context, correlation, and identifying *why* an event happened.

Key Indicators: Logs, Metrics, and Traces (and often Profiling or Events).

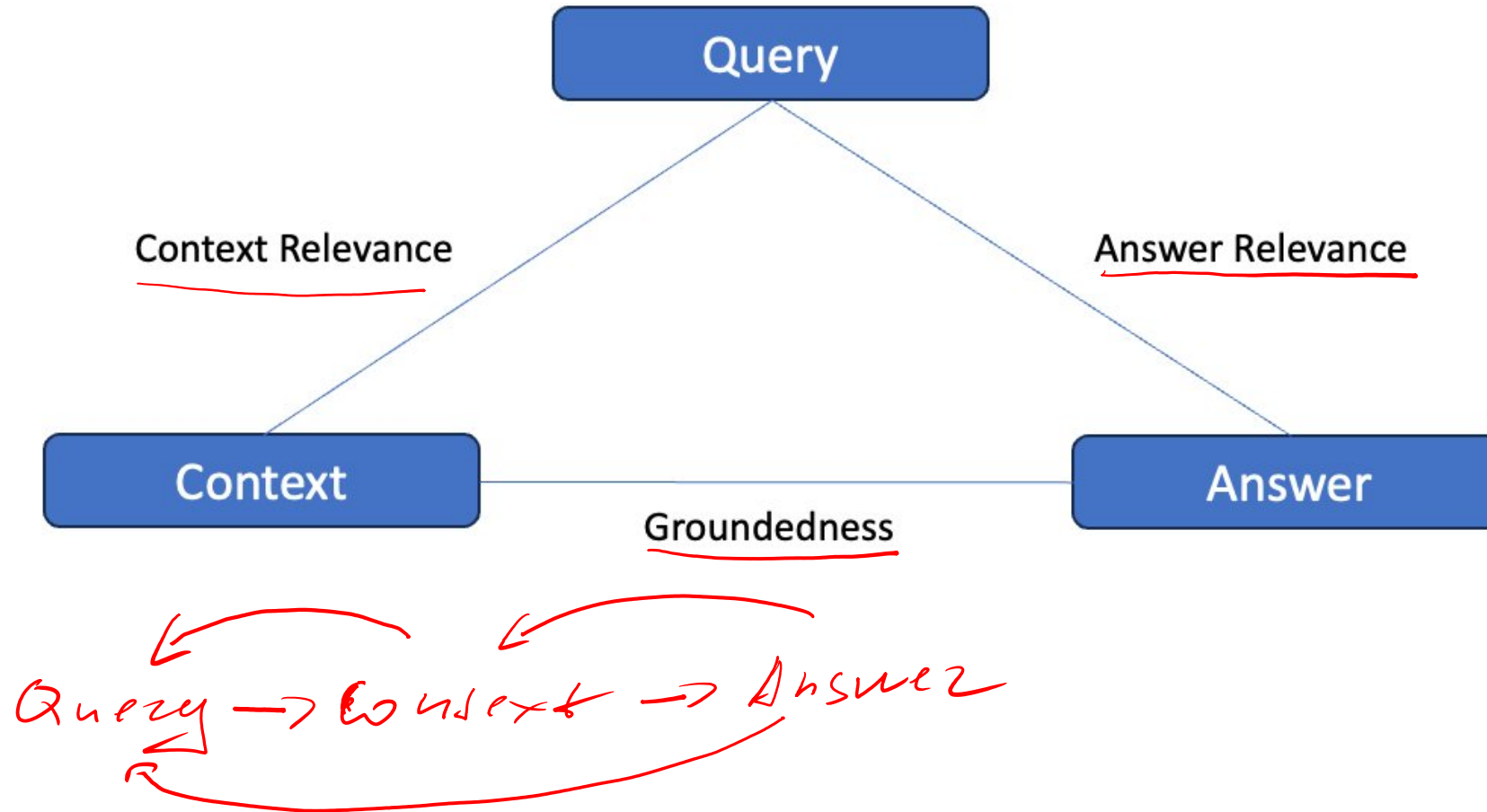
Watching Guardrails is a part of Monitoring process...



Watching Guardrails is a part of Monitoring process... until it's not



Watching RAG triad is a part of Observability



Observability is about building trust with an LLM-powered app

Do you trust an app?

Do your colleagues/supervisors trust an app?

Observability is about building trust with an LLM-powered app

Do you trust an app?

Do your colleagues/supervisors trust an app?

...given:

- LLM vendors often change the base model
- LLMs are not deterministic and therefore prone to hallucinations
- Just because your changes look good on a few examples doesn't mean they are better in general

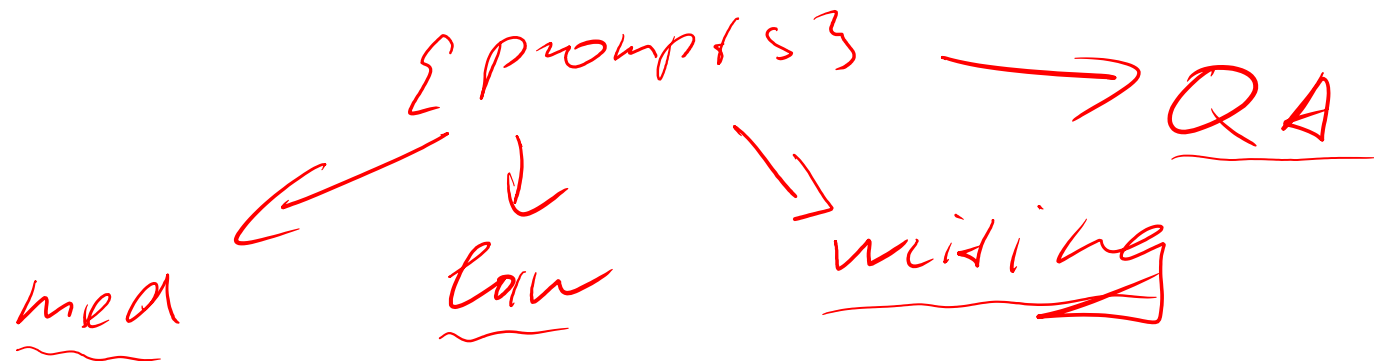
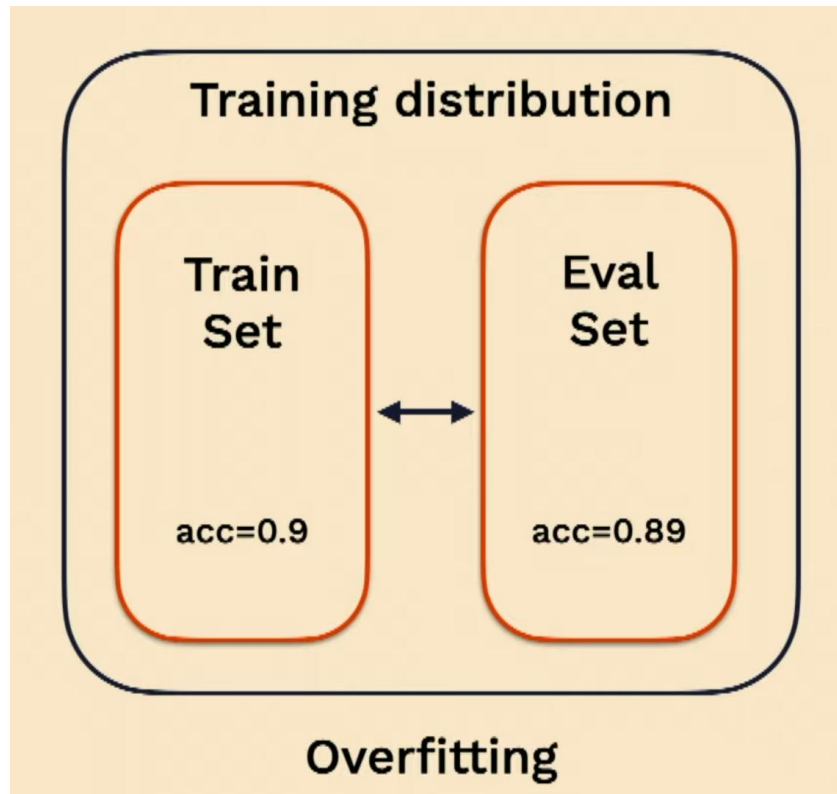


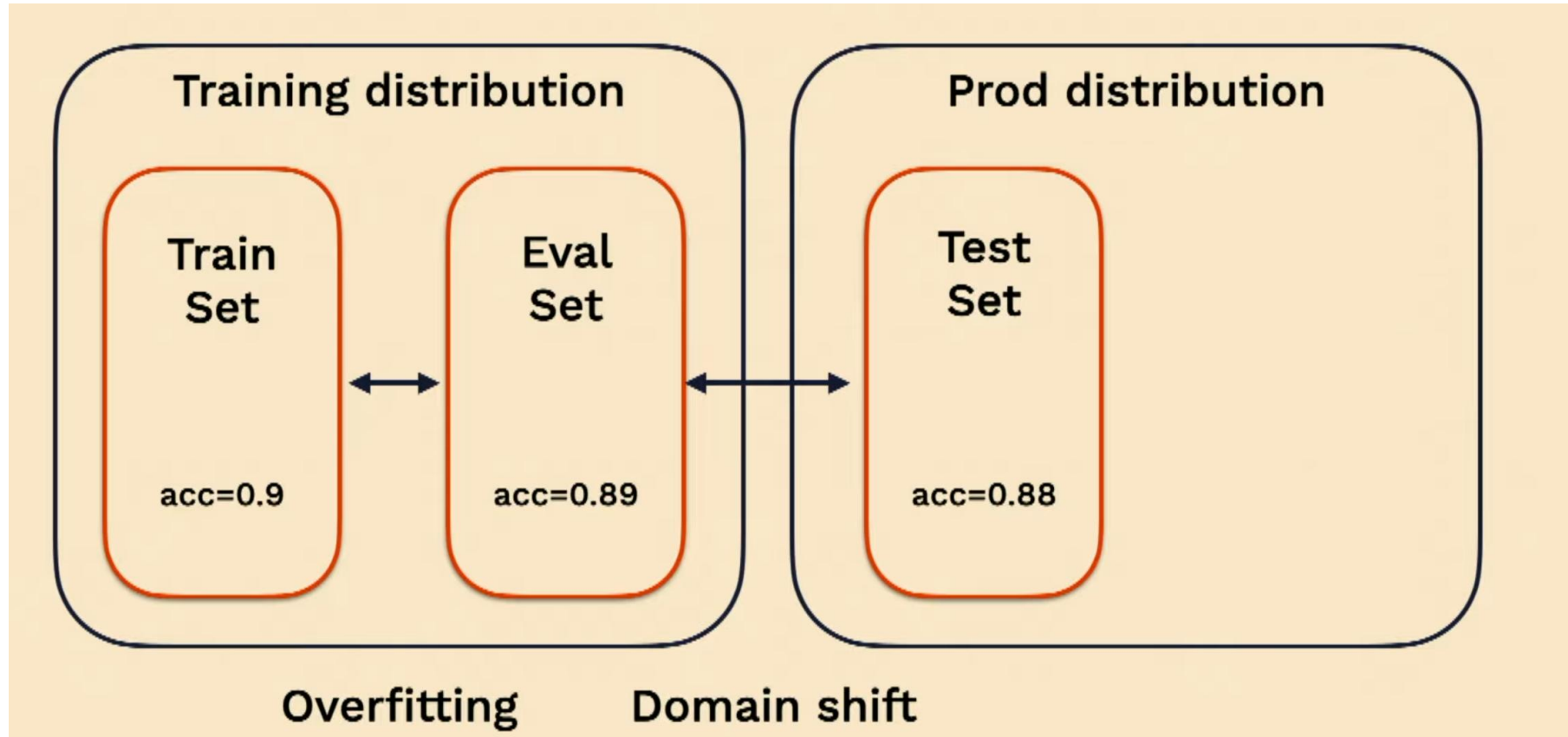
Table of Content

- Monitoring vs. Observability
- **Caveats of “The Old Ways” of testing ML models**
- An overview of ways to test your LLM

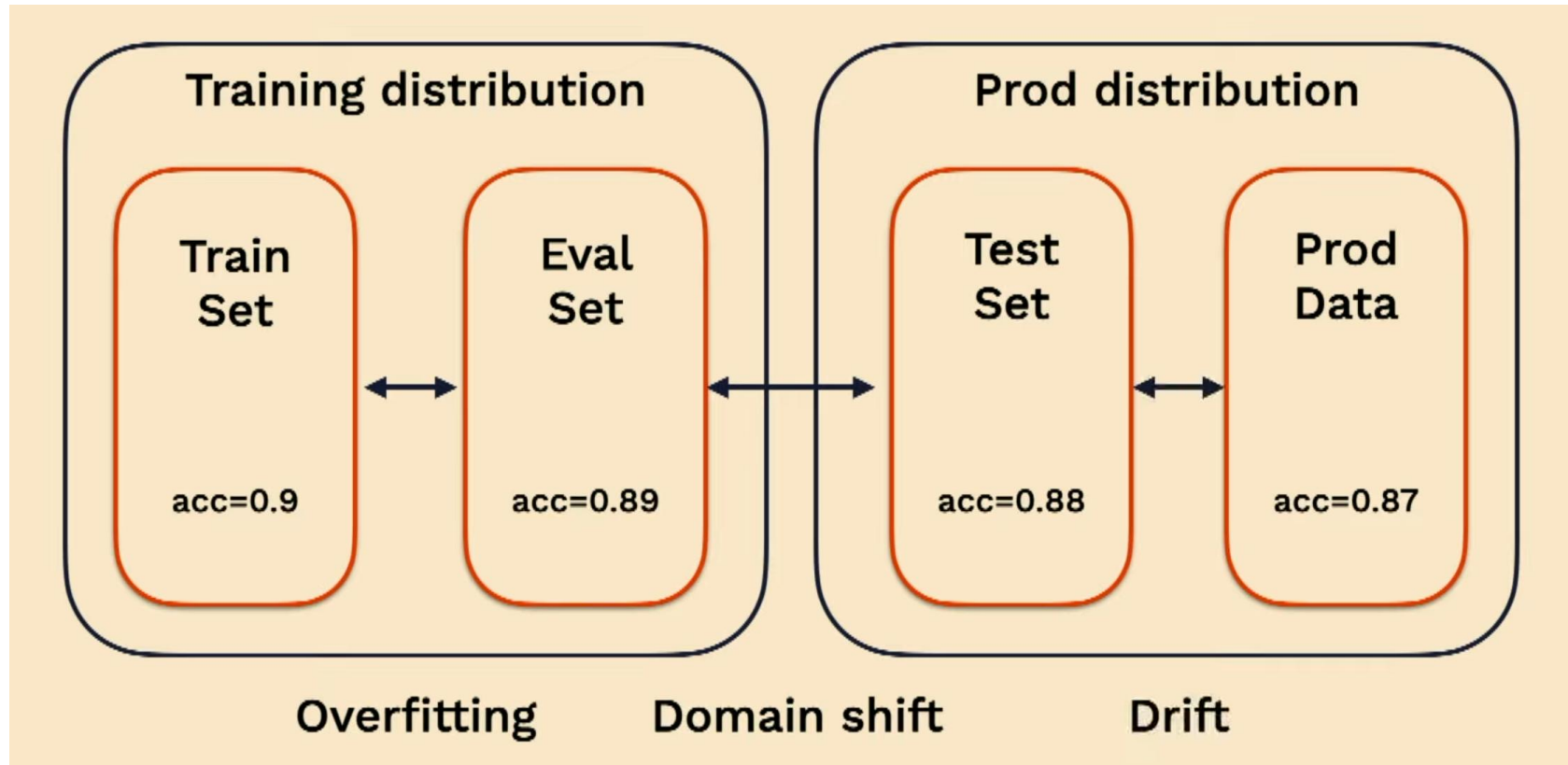
Testing ML models: the old-school way



Testing ML models: the old-school way



Testing ML models: the old-school way



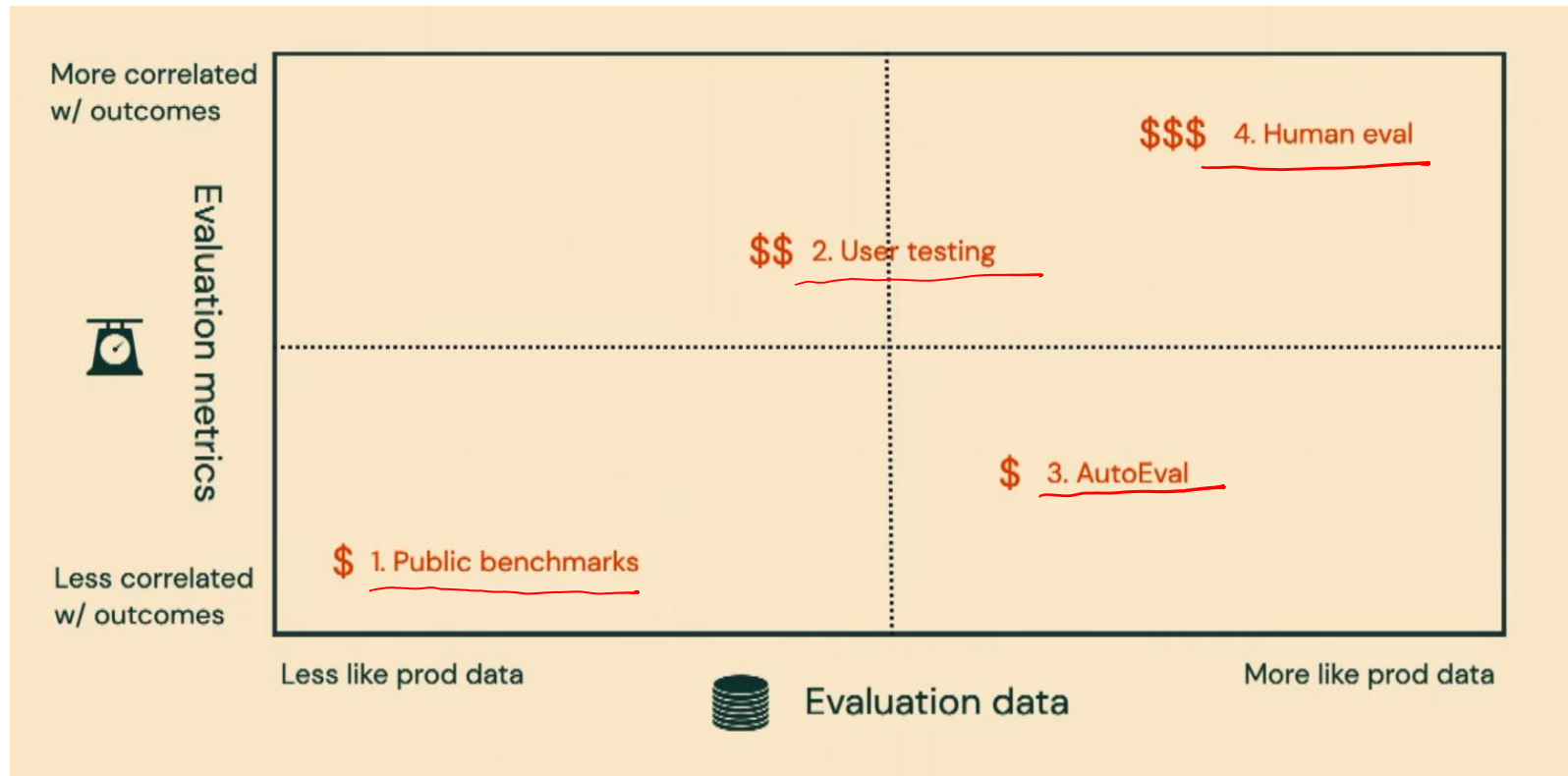
Problems with LLM testing

1. We often do not have access to training data
2. Production data is almost always different from train
3. It's hard to define metrics as they are qualitative
4. LLMs are universal i.e. aggregate metrics don't work
5. etc.

Table of Content

- Monitoring vs. Observability
- Caveats of “The Old Ways” of testing ML models
- **An overview of ways to test your LLM**

A map of means to test your LLM



Tool Calling

- ♥ • BFCL (Berkeley Function Calling Leaderboard) - the most popular benchmark of function calling
- ♥ • ToolBench: Massive tool-use benchmark with thousands of real REST APIs
 - API-Bank: Smaller but clean benchmark, tens of APIs
 - RestBench–TMDB/Spotify: Narrow domain focused benchmarks (video/audio related tools)
 - ToolHop: Purpose-built for multi-hop tool-use: each query requires several interdependent tool calls
- ♥ • GAIA (General AI Assistant benchmark): Realistic assistant tasks (code, web search, data files, multimodal)

Agents

- GAIA (General AI Assistant benchmark): Realistic assistant tasks (code, web search, data files, multimodal)
- ALFWorld: Text-based household environment where an agent completes tasks like “wash the apple and put it in the fridge” via stepwise commands.
- WebShop: Simulated e-commerce site and natural-language shopping instructions like “buy cheap black running shoes with rating > 4.5.”
- τ -bench (Tau-bench): Domain-specific tasks (e.g., airline/retail) with realistic APIs and policies where the agent must converse, ask clarification questions, and call tools.
- τ^2 -bench (Tau²-bench): Extension of τ -bench to dual-control settings where both agent and user have tools and jointly affect the environment state.

Models tend to favor certain order

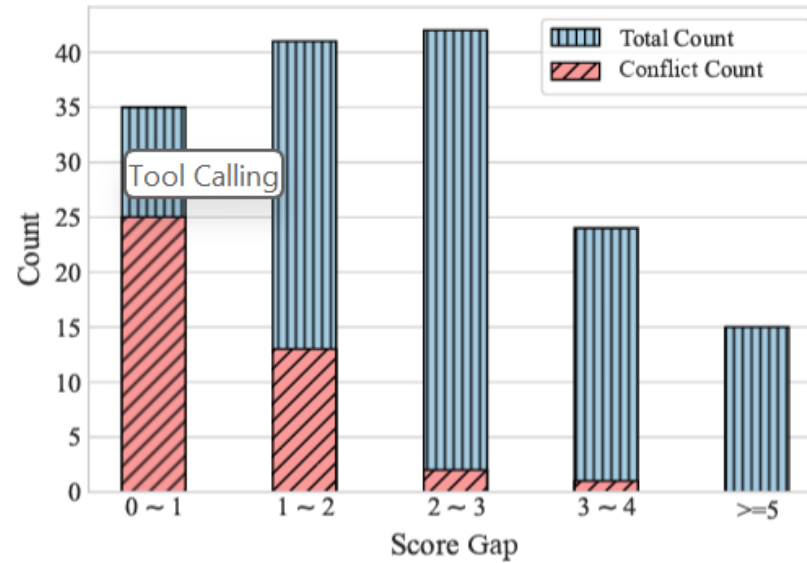


Figure 2: The conflict rate is negatively correlated with the score gap between the two responses. When swapping the order of two responses, the smaller the score gap between them, the more likely GPT-4 is to produce conflicting results.

Humans tend to prefer style, not factuality

Query: How does actor critic improve over REINFORCE?	
ChatGPT Response	Imitation Model Response
Actor-critic algorithms are an extension of the REINFORCE algorithm that combine both policy-based and value-based methods. Some of the changes that actor-critic algorithms make over standard REINFORCE: 1. Function Approximation: Actor-critic algorithms typically use function approximation methods to represent the value and policy functions. This allows them to learn in high-dimensional state spaces. 2. Critic Network: In actor-critic algorithms, there is an additional critic network that estimates the value function. The critic provides an estimate of the value of a state, which is then used to update the policy. This reduces the variance of the update and stabilizes learning. 3. Advantage Estimation: Actor-critic algorithms use the advantage function to estimate the quality of an action taken by the policy. The advantage function is the difference between the return of an action and the estimated value of a state. This reduces the variance of the update and allows for more efficient learning. Overall, actor-critic algorithms provide a more efficient and stable way of learning in reinforcement learning tasks, particularly in high-dimensional state spaces.	Actor-critic algorithms are a type of reinforcement learning algorithm that improves the REINFORCE algorithm by combining the actor (policy) and critic (value) components. The main changes that actor-critic algorithms make over standard REINFORCE are: 1. Actor-critic algorithms use a single critic to evaluate the value function of the policy, whereas standard REINFORCE uses a separate critic for each action. 2. Actor-critic algorithms use a single actor to determine the actions to take, whereas standard REINFORCE uses a separate actor for each action. 3. Actor-critic algorithms learn the policy and actor simultaneously, whereas standard REINFORCE learns the policy and actor separately. 4. Actor-critic algorithms use an optimistic estimate of the value function to determine the actor's actions, whereas standard REINFORCE uses the critic's estimate. Overall, actor-critic algorithms combine the critic's and actor's estimates in order to learn both the policy and actor simultaneously, while standard REINFORCE learns them separately. This can lead to faster convergence and potentially better generalization, but the choice between the two algorithms depends on the specific problem and the desired behavior of the agent.

Figure 2: ChatGPT and our best imitation model produce answers with similar *style*—they start with an overview paragraph, a list of differences, and end with a summary. However, while ChatGPT's answer is mostly correct, the imitation model's answer is *completely* inaccurate despite sounding authoritative. We show correct sentences in green, ambiguously-correct sentences in yellow, and incorrect ones in red.

СПАСИБО ЗА ВНИМАНИЕ